Research Note: What Cube is Teaching Us About Successor Measures

The dataset gives the agent good behavior, but not the task

The OGBench cube-single-play dataset is made of long play trajectories generated by a non-Markovian planning oracle with temporally correlated noise. The oracle repeatedly picks up the single cube, moves it to a random target, and immediately starts a new pick-and-place operation after finishing the previous one. In the official generator, cube-single has no stacking, and the default collection uses 1,000 episodes of 1,001 steps. The evaluation goals are not explicitly demonstrated as labeled tasks; the learner must recover goal-directed behavior from these unstructured trajectories.

This creates an important kind of offline dataset:

- The data contains many competent manipulation actions, but it does not say which action should be used for a new goal.
- It also contains relatively few deliberate failures. The play policy usually forms a workable grasp and continues transporting the cube. Thus, the dataset contains many examples of successful pick-and-place behavior, but much less information about the counterfactual branch where a nearly identical grasp slips or drops.

Our first finding: the hard part begins after reaching the cube

We initially decomposed manipulation into four phases:

$$\text{approach} \rightarrow \text{acquire control} \rightarrow \text{maintain control} \rightarrow \text{transport}.$$

Almost every method reaches the cube, and most methods lift it. The main differences appear afterward: weaker methods drop the cube or fail to steer it to the goal. For the lower-performing methods, roughly 65%–83% of failures occur after the control-acquisition criterion has already been satisfied.

This led to a natural successor-measure hypothesis:

> A good algorithm must distinguish acquired-control states that lead to stable transport from visually similar states that lead to slipping or dropping.

That physical diagnosis remains useful. However, the new experiments show that accurately ranking whole states by future success is not the only route to solving the task.

The surprising value-function result

We compared two GCIQL critics:

$$Q_{\mathrm{MLP}}(s, a, g) = f(s, a, g),$$

and

$$Q_{\mathrm{bil}}(s, a, g) = \phi(s, a)^{\top} \psi(g).$$

The MLP critic is much better at answering:

> From this state, how likely is the agent to eventually succeed?

On in-hand states, the MLP value obtains a success-ranking AUC around 0.81, while the bilinear value obtains only about 0.39. The bilinear critic is therefore not a better calibrated reachability estimator.

Yet the bilinear policy is much better. To understand this, we decomposed each critic into

$$Q(s, a, g) = U(s, g) + A(s, a, g),$$

where

$$U(s, g) = \mathbb{E}_{\tilde{a} \sim \mu(\cdot|s)} Q(s, \tilde{a}, g)$$

is the action-independent baseline and

$$A(s, a, g) = Q(s, a, g) - U(s, g)$$

is the action-dependent preference.

The MLP's strong state ranking lives almost entirely in $U(s, g)$. The bilinear baseline is badly calibrated, but its useful information is concentrated more in the action-dependent part. Quantitatively, the action-independent baseline $U = \mathbb{E}_{a \sim \mu} Q$ alone ranks in-hand success at AUC 0.82 for the MLP but only 0.40 for the bilinear critic — the bilinear baseline is not merely weak but anti-calibrated. Across states the two baselines are essentially uncorrelated (Spearman $\rho(U_{\mathrm{bil}}, U_{\mathrm{MLP}}) = -0.04$), yet within a state their action-dependent parts partially agree ($\rho(A_{\mathrm{bil}}, A_{\mathrm{MLP}}) = +0.22$ over a shared μ action set): orthogonal value geometry, shared policy geometry.

This matters because a policy does not need to compare the value of unrelated states. At a fixed state, it only needs to choose between actions:

$$\arg\max_a Q(s, a, g) = \arg\max_a A(s, a, g).$$

An arbitrary state-goal offset can be wrong without changing the chosen action.

The decisive experiment: use the critic itself as the policy

At every step, we sampled 32 plausible actions from a frozen behavior-cloning flow,

$$a_1, \ldots, a_{32} \sim \mu(\cdot \mid s),$$

and executed

$$a^{\star} = \arg\max_i Q(s, a_i, g).$$

Policy	Success
Behavior flow without reranking	≈ 0.06
MLP critic reranking	0.57–0.59
Bilinear critic reranking	0.91–0.96
Trained bilinear flow actor	0.947

No actor was trained. The frozen critic simply reranked behavior-model actions.

The result was:

The result persists when the candidate actions are sampled from either behavior flow (bilinear-μ candidates: 0.96 vs 0.59; MLP-μ candidates: 0.91 vs 0.57). Thus, it is not caused by a lucky candidate generator. The final bilinear critic itself contains a nearly complete Cube policy in its ordering of behavior-supported actions.

Sweeping the candidate count $N \in \{1, \ldots, 128\}$ separates coverage from ranking. Bilinear reranking rises to 0.95 by $N = 32$ — reaching its trained actor (0.933) — so argmax-over-behavior alone recovers the policy; it is ranking, not coverage, that carries it. (Beyond $N = 32$ it declines slightly, to 0.85 at $N = 128$, as the larger candidate set begins to expose the critic's off-support optimistic peaks.) MLP reranking instead saturates near 0.6 even at $N = 128$ and stays *below* its own trained actor (0.747): for the MLP, ranking is the bottleneck and continuous action modification matters. The critic-field advantage is not a co-adaptation artifact either: a *fresh* flow actor trained against the *frozen* bilinear critic reaches ≈ 0.88, versus ≈ 0.68 against the frozen MLP critic — the same $\sim$0.2 gap, with no shared training history.

The simplest interpretation is:

- the behavior flow is an action library;
- the bilinear critic is a goal-conditioned retrieval rule;
- the trained flow actor is mostly an amortized version of this retrieval.

Ordinary behavior cloning fails because it knows which actions are plausible, but not which plausible action matches the current goal.

Why flow works and DDPG+BC does not

The bilinear critic is reliable only under a particular contract:

> Compare actions that look like actions from the offline dataset.

The flow actor respects this contract. It remains anchored near the behavior distribution.

DDPG+BC instead optimizes Q continuously in action space. With the bilinear critic, it moves far away from the behavior-action cloud (whitened nearest-neighbour departure ≈ 17.9 vs 1.76 for the MLP critic) and finds actions that the critic scores above *every* sampled dataset-like action 79% of the time (median gap +2.14; the MLP does so only 6% of the time). It is a genuine off-support maximum, not a globally steeper landscape — the action-gradient norms are equal (3.18 vs 3.06). A fresh DDPG+BC actor trained against a frozen bilinear critic shows the same failure, reaching only ≈ 0.52 (matching the co-trained bilinear–DDPG+BC at 0.557, and far below the flow actor's 0.88), so this is not merely a co-training artifact — the peak lives in the frozen critic geometry.

The resulting picture is:

inside the behavior region: Q_{bil} orders actions extremely well;
outside the behavior region: Q_{bil} has unreliable optimistic peaks.

Flow or finite-sample reranking uses the first property and avoids the second.

What this says about successor measures

A successor measure describes the full discounted distribution of future states:

$$M(s, a, \mathrm{d}s').$$

A reward gives a value through

$$Q_g(s, a) = \int r_g(s') M(s, a, \mathrm{d}s').$$

But a policy does not need the complete future distribution. It only needs to know how the future changes when the action changes.

Using the behavior distribution μ, define the average successor measure

$$\bar{M}^{\mu}(s, \mathrm{d}s') = \mathbb{E}_{a \sim \mu(\cdot|s)} M(s, a, \mathrm{d}s'),$$

and the action-centered successor difference

$$\Delta M^{\mu}(s, a, \mathrm{d}s') = M(s, a, \mathrm{d}s') - \bar{M}^{\mu}(s, \mathrm{d}s').$$

Then

$$Q_g(s, a) = U_g^{\mu}(s) + A_g^{\mu}(s, a),$$

where

$$U_g^{\mu}(s) = \int r_g(s') \bar{M}^{\mu}(s, \mathrm{d}s')$$

and

$$A_g^{\mu}(s, a) = \int r_g(s') \Delta M^{\mu}(s, a, \mathrm{d}s').$$

Only the second term affects action choice:

$$\arg\max_a Q_g(s, a) = \arg\max_a A_g^{\mu}(s, a).$$

This is the central lesson:

> A standard successor-measure objective learns the whole future. Cube control mainly needs the action-dependent difference between futures.

Why full successor-measure learning can miss this difference

In the Cube play dataset, much of the future is shared across plausible actions:

- the arm remains near the same workspace;
- the rollout remains in the same manipulation phase;
- the scripted policy usually continues a successful pick-and-place operation;
- unstable drop branches are relatively uncommon.

Therefore, two actions can have very similar total future occupancy while differing in the small part that determines control:

- one centers the grasp;
- one creates marginal contact;
- one preserves the cube during acceleration;
- one eventually causes a drop.

A full-measure reconstruction loss can be dominated by the large common component $\bar{M}^\mu$. The small action-dependent correction ΔM^μ may contribute little to average prediction error even though it completely determines the policy.

This explains how a successor-measure method can fit future occupancy reasonably well yet choose the wrong action. It also explains why improving a measure residual need not improve Cube success.

In exact arithmetic with unlimited capacity, a correct successor measure contains everything needed for control. The problem is finite capacity, finite data, and the training objective: learning the stronger object does not guarantee that the limited representation preserves the smaller policy-critical component.

What a successor-measure algorithm may need to crack Cube

A successful successor approach should probably contain four pieces.

1. Learn action-centered successor differences

Instead of spending all capacity on

$$M(s, a, \cdot),$$

explicitly represent

$$\Delta M^\mu(s, a, \cdot) = M(s, a, \cdot) - \mathbb{E}_{\tilde{a}\sim\mu} M(s, \tilde{a}, \cdot).$$

This is the successor-measure analogue of an advantage function.

2. Train for action ordering, not only occupancy reconstruction

For two plausible actions a_i and a_j, the control-relevant quantity is

$$Q_g(s, a_i) - Q_g(s, a_j) = \int r_g(s') \left[M(s, a_i, \mathrm{d}s') - M(s, a_j, \mathrm{d}s') \right].$$

The algorithm should directly preserve the sign and margin of these pairwise differences.

3. Restrict policy extraction to supported actions

Use a behavior model as a proposal distribution:

$$a_i \sim \mu(\cdot \mid s), \qquad a^\star = \arg\max_i Q(s, a_i, g).$$

This avoids asking a finite-data successor model to extrapolate reliably over arbitrary actions.

4. Separate value geometry from policy geometry

A natural architecture is

$$Q(s, a, g) = U_{\mathrm{MLP}}(s, g) + A_{\mathrm{succ}}(s, a, g),$$

where:

- U_{MLP} models calibrated global reachability;
- A_{succ} models action-centered successor differences;
- the actor consumes only A_{succ}.

This allows the method to retain the global planning benefits of successor measures while allocating explicit capacity to the local action comparisons that Cube requires. We have implemented this architecture directly on GCIQL (a *behavior-centered dueling critic*: $A_{\mathrm{bil}} = (\phi(s, a) - b(s))^\top \psi(g)/\sqrt{d}$, with $b(s)$ a learned head regressed onto the behavior-action features so that $\mathbb{E}_{a\sim\mu} A_{\mathrm{bil}} \approx 0$, a distinct expectile value head, and the flow actor consuming only A_{bil}); its Cube evaluation is in progress.

The revised interpretation of the original diagnostics

Our original paper asked whether values distinguish stable in-hand states from unstable ones. That remains a useful diagnostic, because many native successor methods fail mainly after acquiring the cube.

The bilinear result shows, however, that cross-state value ranking is not necessary for excellent control. We now need three separate questions:

- **State-value fidelity:** Does the method know which state is more promising?
- **Within-state action fidelity:** Does it know which plausible action is better at the current state?

- **Extractability:** Can the policy use that ordering without leaving the region supported by data?

The bilinear critic is poor on the first, excellent on the second, and requires flow or reranking for the third.

The immediate successor-measure experiment is therefore simple:

$$a^\star = \arg\max_{a_i \sim \mu} Q_{\mathrm{FB/SF}}(s, a_i, g).$$

If FB or SF remains weak under behavior-sample reranking, then its training objective failed to learn the policy-relevant successor differences. If reranking makes it strong, then the successor representation was adequate and policy extraction was the real bottleneck.

The same idea in the simplest possible RL language

The offline dataset contains many good robot actions. Behavior cloning learns:

> These are actions the robot commonly takes.

It does not learn:

> For this goal, take this particular action.

The bilinear critic learns to sort the plausible actions:

> sample plausible actions → score them for the goal → take the best one.

A successor-measure method usually tries to predict everything that will happen in the future. That may waste capacity on parts of the future that are the same for every action. For control, the important question is simpler:

> How will the future be different if I take action a_1 instead of action a_2?

To solve Cube, a successor-measure algorithm should learn those differences between action futures, use them to rank dataset-supported actions, and avoid maximizing the model far outside the offline data.